

人工智能基础与应用 · 大模型实训

M4-M9 · 展开大纲（按周写完整版）

从零实现大模型：M0-M9 知识主干（16 周 / 11 课题 / 零预算）

孙承泽 · 能动强基 2501 · 西安交通大学

2026-09-17 生成 · 版本 v1.0

Markdown 母版：人工智能基础与应用/讲义/ 下的 .md 文件 · 配套上游：CS336 A1-A5 · nanochat · minimind

讲义 M4-M9 · 展开大纲（按周写完整版）

说明：本文件是 M4-M9 的**骨架大纲**（核心问题 + 关键概念 + 必读 + 实验 + 坑），每周讲课前我会把当周讲义**展开成完整版**（新增 M4-训练.md 等文件），不留空讲义占位。这样做的理由：这些模块严重依赖课题 04 起的**真实实验数据**，提前写死会出现“讲义与实测不符”。

M4 · 训练：数据、优化、系统（W3-W9）

要回答的问题：为什么同一个模型有人训得动有人训崩？scaling law 怎么指导算力分配？多卡为什么不是线性加速？

关键概念：预训练目标与数据配比 | 过滤与去重 | AdamW（解耦权重衰减的意义）| Muon | LR schedule（warmup + cosine）| 梯度裁剪 | 混合精度（bf16 vs fp16 vs fp32, GradScaler 何时必需）| 梯度累积 | DDP / FSDP / TP / PP | Chinchilla 的 compute-optimal | `--target_flops` 与“固定算力预算下的最优 (N, D)”

必读：CS336 lecture 05-11 + A2/A3 handout · nanochat/optim.py、base_train.py、runs/scaling_laws.sh · d2l ch11-ch12

实验：手写 AdamW 与调度 → 单卡固定算力下的 depth sweep（d4/d6/d8/d10）→ 拟合幂律 → **先写死外推预测** → 跑 d12 验证（打脸链路）

坑：把“loss 不降”当数据问题（常是 LR/初始化/精度）| 把 DDP 当免费加速（PCIe 下扩展效率可能只有 1.3-1.5×）| T4 无 bf16（必须 fp16 + GradScaler）| 显存不足先怀疑 batch 与精度，再怀疑硬件

M5 · 推理与部署（W5-W8）

要回答的问题：为什么生成必须逐 token？KV cache 省了什么、代价是什么？量化掉多少点？吞吐与延迟的取舍？

关键概念：自回归解码 | **KV cache**（显存公式见 M3 § 五）| continuous batching | PagedAttention | 采样策略（temperature / top-p / top-k / repetition penalty）| 量化（int8/int4/GPTQ/AWQ）| 投机解码 | 每 token 成本

必读：nanochat/engine.py（KV cache 实现）+ infer_bench.py · CS336 lecture 10 · rasbt《Reasoning》ch2

实验：从零写带 KV cache 的生成 → 测有/无 cache 的耗时与显存 → 贪心 vs top-p 对照 → batch size 对吞吐的影响

坑：以为“慢是因为模型大”（常是没批处理/没 cache）| 以为量化无损 | 忽略 prefill 与 decode 两个阶段特性完全不同（前者算力密集，后者显存带宽密集）

M6 · 后训练：从“会说话”到“会做事”（W10-W12）

要回答的问题：SFT / DPO / RLVR 各补什么能力？为什么 RLVR 提数学/代码却可能伤通用能力？reward hacking 长什么样？

关键概念：指令微调与 chat template | **损失掩码**（只算 answer 部分）| 奖励模型 | PPO 的四个模型 | DPO 的隐式奖励（推导：为什么可以不要 reward model）| **GRPO 的组内归一化优势** | KL 正则 | 拒绝采样 | 蒸馏与合成数据 | 过度优化（over-optimization）

必读: CS336 A5 (含 tests/test_grpo.py、GSPO/MaxRL) · rasbt《Reasoning from Scratch》ch3–ch8 · Nathan Lambert《RLHF Book》(免费在线, 17 章) · minimind/trainer/train_grpo.py · nano-aha-moment (单文件)

实验: 同基座做 SFT / DPO / GRPO 三变体 → 同一评测口径对照 → 手写 GRPO 损失并逐项注释 → **制造一次 reward hacking 并记录**

坑: 以为 DPO “更简单所以更好” | 以为 RL 一定比 SFT 强 | **把训练 reward 上升当能力提升** (不做 held-out 评测就是自欺) | KL 系数设错导致模型 “复读训练集答案”

M7 · 评测: 把 “感觉变好了” 变成 “确实变好了” (W9–W12)

要回答的问题: benchmark 分数可信吗? 污染怎么判断? 为什么必须固定口径? 方差有多大?

关键概念: bpb / PPL / CORE | 下游基准 (GSM8K / MATH-500 / MMLU / HumanEval) | pass@k | 评测污染 | few-shot 与 prompt 敏感性 | 方差与误差棒 | LLM-as-judge 的偏置 | 可验证奖励 vs 学习型奖励

必读: rasbt ch3 (先建评测再上技术——全书教学法基石) · nanochat/core_eval.py (单文件 CORE) · CS336 lecture 12 · Berkeley Agentic AI MOOC 的 “Agent Evaluation” 讲

实验: 写一个数学题判定器 (解析 + 数值比对 + 容错) → 测它在 200 题上的准确率与**假阳性** → 给两个模型跑同一套评测并给误差棒

坑: 只看一个 benchmark 下结论 | 忽略 prompt 格式造成的巨大差异 | 把测试集当验证集反复调 | 只报均值不报方差

M8 · 智能体: 工具、记忆、harness、长程任务 (W13–W14)

要回答的问题: 从 “回答问题” 到 “完成任务” 缺什么? 为什么 harness 比模型本身更决定成败? self-judge 何时有效、何时自欺?

关键概念: 工具/函数调用 | ReAct 循环 (Thought-Action-Observation) | **context engineering** (李宏毅与 Karpathy 都专讲) | 记忆与压缩 | harness 设计 (沙箱、权限、反馈信号) | 长程任务的误差累积 | 多智能体 | self-judge / self-refine 的边界 | 可验证环境

必读: 李宏毅《Context Engineering》专讲 · Berkeley LLM Agents / Agentic AI MOOC (含 “Post-Training Verifiable Agents”) · HF Agents Course (三框架 + 公开排行榜) · rasbt ch5 (self-refinement) · PrimeIntellect-ai/verifiers · terminal-bench

实验: 手写最小 ReAct 循环 (不用框架) → 给两个工具 (计算器 + 代码执行) → 记录失败轨迹并分类 → 加 self-judge 做消融

坑: 以为瓶颈在模型智力 (常在你的工具接口与反馈信号) | 以为 self-judge 免费提升 (**它会放大原有错误**) | 上下文不做压缩导致长任务后段 “忘记目标”

M9 · 前沿与安全 (W15–W16 选题制)

要回答的问题: MoE / 长上下文 / 多模态改变了什么? 对齐与安全是 “加一层” 还是 “改训练目标”? “AI 训 AI” 可行到什么程度?

关键概念: MoE 稀疏激活 (总参数 vs 激活参数) | 长上下文外推 | 多模态对齐 | 可解释性 | 红队与越狱 | 幻觉的成因 | 合成数据闭环 | self-play | 灾难性遗忘 | 模型合并

必读: CS336 lecture 04 (MoE) · opendilab/awesome-RLVR · LeapLabTHU/limit-of-RLVR · 论文精读清单联动

实验（三选一）：合成数据自训练一轮 / 拒绝采样提升数学 / self-play 生成题目 → 做最小闭环 + **明确写出这个方向的 no-go**

坑：把“前沿”当必须立刻掌握（正确姿势是**保持索引能力 + 每季度更新一次本模块**） | 只看结论不看评测口径 | 把“能跑通”当成“有结论”

展开时机表

讲义文件	需要先有	预计展开时间
M4-训练.md	课题 02 收口（能算 FLOPs/显存）	W3
M5-推理.md	课题 04 首轮训练成功（有模型可推理）	W5
M7-评测.md	有第一个模型输出	W9
M6-后训练.md	课题 07 数据管线 + 评测口径固定	W10
M8-智能体.md	有 SFT 后的模型	W13
M9-前沿.md	全部证据齐备	W15